

Predicting Ordering Intent Through Enhanced User Interaction Signals

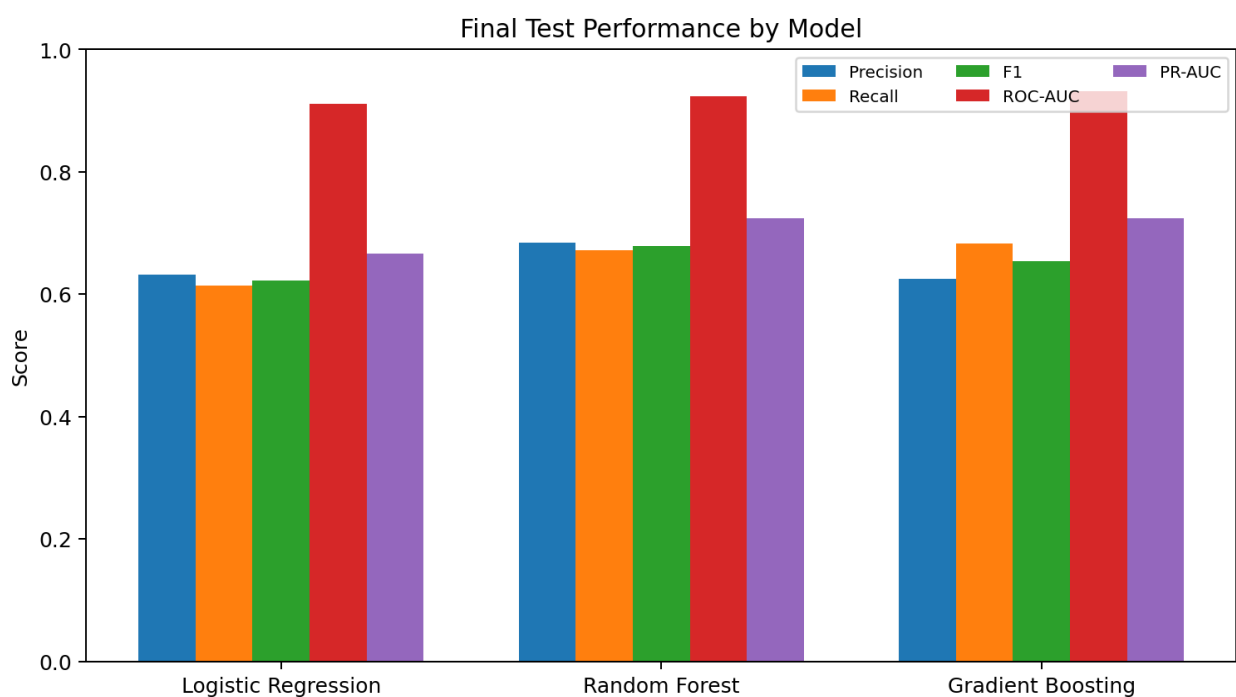
Objective: Build and evaluate classification models that predict whether an online shopping session results in a purchase, using session-level user interaction signals.

Dataset: Online Shoppers Intention dataset. The notebook starts with 12,330 rows and 18 columns, removes duplicates, and works with 12,205 unique rows and 17 predictor columns plus the binary target *MadePurchase*.

Models: Logistic Regression, Random Forest, and Gradient Boosting.

Evaluation design: 60% train, 20% validation, 20% test with stratification. Classification thresholds are selected on validation data and then locked before the final test evaluation.

Important data-quality note: The notebook finds no missing values in the observed dataset. Imputation remains in the pipelines as a defensive preprocessing step.



The final test results show that the models have broadly strong ranking performance, while threshold choice changes the precision-recall trade-off for purchase detection.

1. Problem Definition

The task is a binary classification problem: predict *MadePurchase* from session-level interaction and contextual variables. The positive class represents a purchase and is less frequent than the non-purchase class.

The original target distribution is 84.53% no purchase and 15.47% purchase. After duplicate removal, the modeling dataset contains 12,205 rows. Because the positive class is relatively uncommon, the notebook uses stratified splitting and class-balancing techniques rather than relying on accuracy alone.

The practical goal is not only to fit a model, but to establish a reproducible evaluation workflow in which threshold selection is separated from final testing.

2. Data Preparation

Duplicate handling: 125 duplicate rows are removed, reducing the dataset from 12,330 to 12,205 rows.

Missing values: The observed dataset has zero missing values across all columns. Median imputation for numeric variables and most-frequent imputation for categorical variables are still included in the pipelines for robustness if future data contains missing entries.

Feature groups: 10 numeric features and 7 categorical features are used. Categorical variables include VisitMonth, UserOS, UserBrowser, UserRegion, SourceChannel, UserCategory, and IsWeekendVisit.

AvgPageValue: The notebook records a correlation of 0.4919 with the target and retains this variable by default. This is not automatically evidence of leakage_ It should remain only if the business prediction setup confirms that the value is available before the event being predicted

3. Methodology

Data split: 60% training (7,323 rows), 20% validation (2,441 rows), and 20% test (2,441 rows), using stratification and random_state=42.

Logistic Regression preprocessing: numeric median imputation plus StandardScaler; categorical most-frequent imputation plus OneHotEncoder(handle_unknown="ignore").

Tree-model preprocessing: numeric median imputation; categorical most-frequent imputation plus one-hot encoding.

Class imbalance: Logistic Regression and Random Forest use class_weight="balanced". Gradient Boosting uses balanced sample weights generated with compute_sample_weight.

Threshold selection: thresholds from 0.10 to 0.90 are evaluated on the validation set, and the threshold that maximizes F1 is selected independently for each model.

Final testing: after thresholds are locked, the untouched test set is evaluated once. The same final predictions are reused for classification reports, confusion matrices, and subgroup analysis.

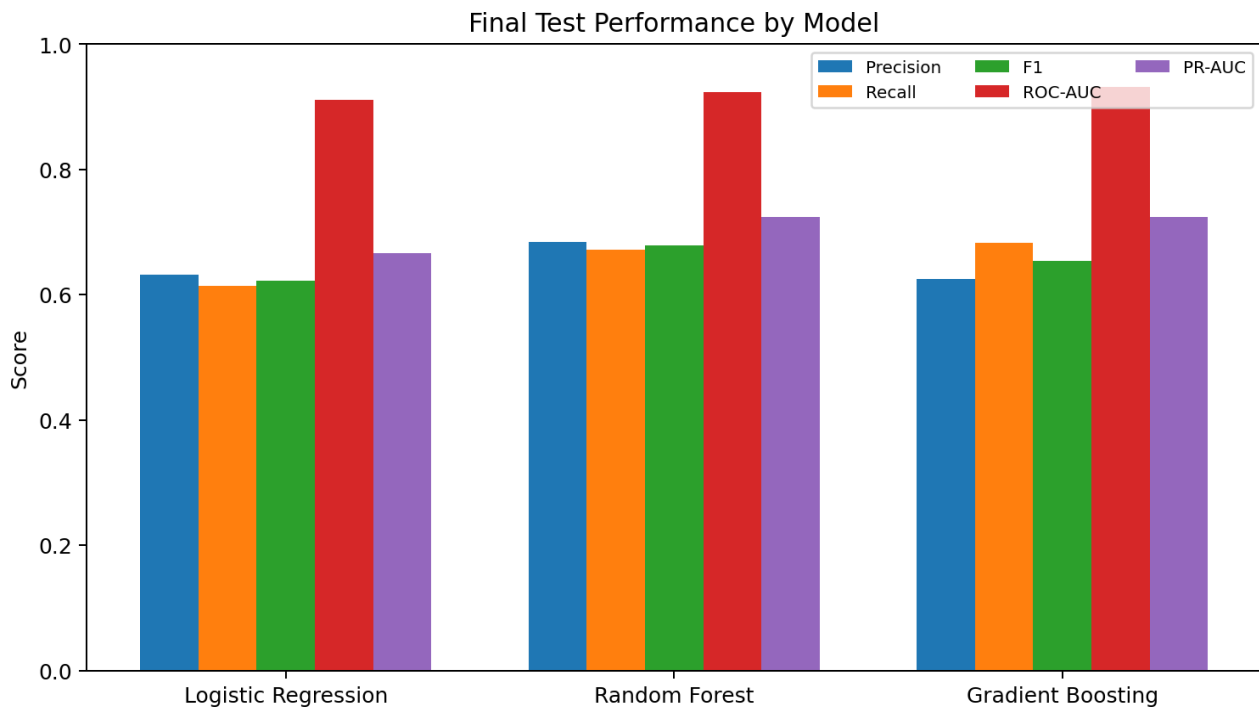
Selected thresholds

Model	Validation threshold	Validation recall	Validation F1
Logistic Regression	0.63	62.73%	62.16%
Random Forest	0.40	63.25%	64.87%
Gradient Boosting	0.68	71.65%	65.55%

4. Final Test Results

The following metrics are computed on the untouched test set after threshold selection on validation data.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Threshold
Logistic Regression	88.78%	63.17%	61.34%	62.24%	0.9110	0.6659	0.63
Random Forest	89.96%	68.46%	67.19%	67.82%	0.9229	0.7240	0.40
Gradient Boosting	89.06%	62.53%	68.29%	65.38%	0.9322	0.7246	0.68



Result interpretation: Random Forest has the highest test accuracy, precision, and F1 among the three models at the selected threshold. Gradient Boosting has the highest ROC-AUC and the highest PR-AUC by a small margin over Random Forest. Logistic Regression provides a strong linear baseline with lower F1 than the two tree-based approaches.

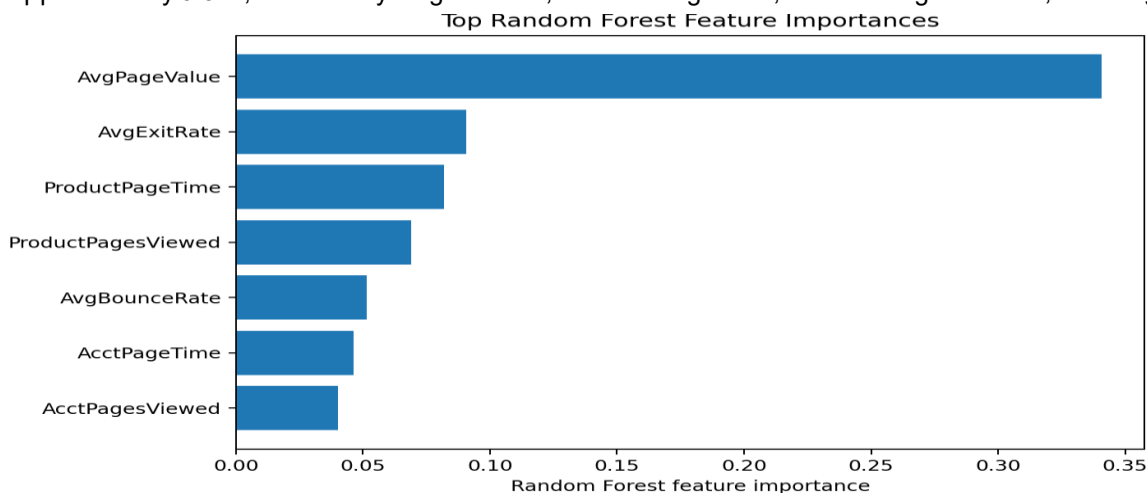
5. Confusion Matrix Summary

Model	TN	FP	FN	TP
Logistic Regression	1908	151	123	259
Random Forest	1942	117	128	254
Gradient Boosting	1887	172	95	287

For purchase prediction, false negatives represent purchase sessions that the model fails to identify, while false positives represent non-purchase sessions predicted as purchases. The selected thresholds determine this trade-off.

6. Feature and Relationship Insights

Random Forest feature importance: AvgPageValue is the largest model-specific importance in the notebook at approximately 0.341, followed by AvgExitRate, ProductPageTime, ProductPagesViewed, and AvgBounceRate.



These importances indicate how the fitted Random Forest uses encoded features to reduce impurity. They should not be interpreted as causal effects, and categorical variables are shown after one-hot encoding.

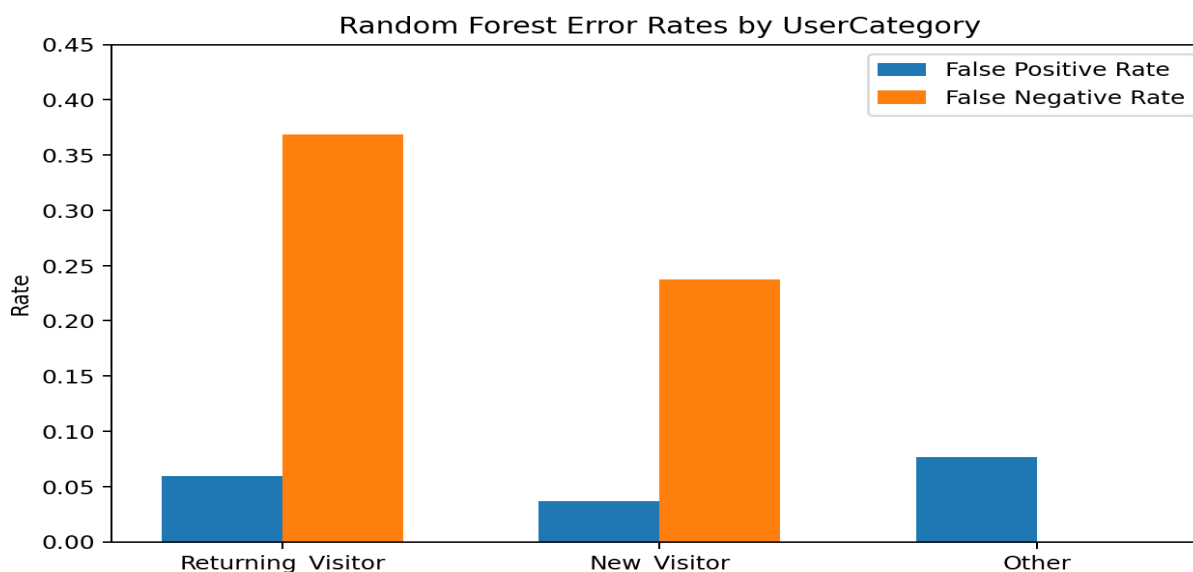
Correlation structure: AvgBounceRate and AvgExitRate have a correlation of approximately 0.902. ProductPagesViewed and ProductPageTime have a correlation of approximately 0.860. These relationships indicate that some behavioral measures carry overlapping information.

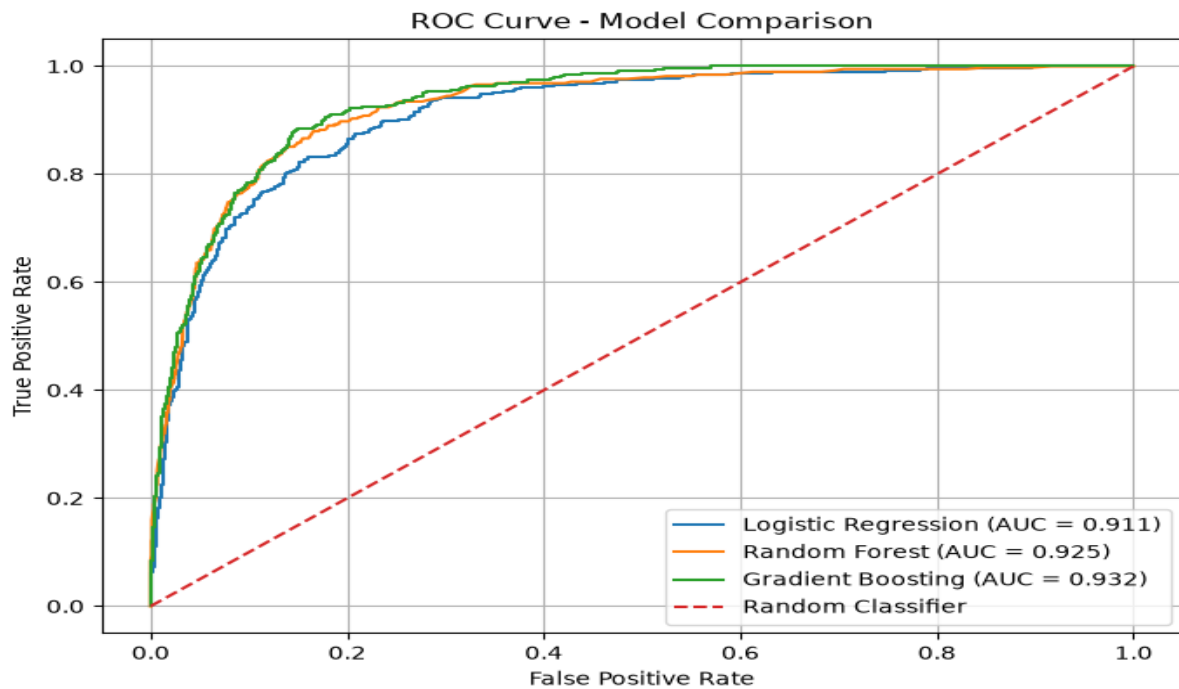
Target association: AvgPageValue has a correlation of approximately 0.492 with MadePurchase in the cleaned dataset. The notebook explicitly flags the need for a temporal availability audit before treating this feature as safe for production use.

7. Subgroup Performance Analysis

The notebook evaluates Random Forest error rates by UserCategory. These are dataset segments and are not automatically protected demographic attributes. Small groups can make subgroup metrics unstable.

UserCategory	Samples	False Positive Rate	False Negative Rate
Returning_Visitor	2,084	5.95%	36.84%
New_Visitor	344	3.64%	23.71%
Other	13	7.69%	0.00%





8. Limitations and Production Considerations

- 1. Temporal feature audit:** AvgPageValue must be verified against the exact prediction timestamp. If its calculation uses post-purchase information, it should be removed before production modeling.
- 2. Hyperparameter tuning:** The notebook compares fixed model configurations; it does not perform a full hyperparameter search or cross-validation-based model optimization.
- 3. Threshold objective:** F1 is used to select thresholds. A production system may need a business-specific objective such as conversion value, lead capacity, or a minimum recall/precision constraint.
- 4. Subgroup stability:** Several dataset categories have very small counts. Additional data or confidence intervals would be appropriate before drawing operational conclusions from subgroup comparisons.
- 5. External validation:** Results come from a single held-out split. A future release should consider repeated cross-validation and a temporally separated holdout when the deployment setting is time-dependent.

9. Conclusion

The updated notebook establishes a cleaner machine-learning evaluation pipeline for online purchase-intent prediction. It separates training, threshold selection, and final testing; uses preprocessing pipelines; addresses class imbalance; and keeps the final analysis consistent with the locked test predictions.

On the final test set, the Random Forest obtains 89.96% accuracy and 0.6782 F1, while Gradient Boosting obtains the highest ROC-AUC (0.9322) and PR-AUC (0.7246). These results describe performance on this dataset and split; they should not be treated as production guarantees without the temporal audit, broader validation, and business-threshold review described above.

10. Reproducibility

Notebook: `ml(1)_updated.ipynb`. Dataset: `online_shoppers_intention.csv`. The notebook accepts the dataset under `data/online_shoppers_intention.csv` and falls back to the project root. Random seed: **42**.